Projet : CatOpt

Samir Gueblaoui

9 novembre 2026

Table des matières

1 Introduction **2**

2 Première version **2**

1 Introduction

L'objectif de ce projet est d'explorer les concepts de la théorie des catégories pour implémenter des optimisations de programmes justifiées par des lois algébriques et catégoriques.

2 Première version

La première version est assez simple : on part d'une catégorie $\mathscr{C}$ dont l'unique objet est l'ensemble $\mathscr{U} = \mathbb{Z}/2^{64}\mathbb{Z}$. Ses morphismes sont les fonctions $\mathscr{U} \to \mathscr{U}$ engendrées par composition usuelle dans **Set** des fonctions $A_c : x \mapsto x + c$, pour $c \in \mathscr{U}$, et $S : x \mapsto x^2$, ainsi que l'identité $\mathrm{id}_{\mathscr{U}} : x \mapsto x$.

Remarque 2.1. *On utilise $\mathscr{U}$ pour modéliser l'arithmétique du type* `std::uint64_t` *utilisé dans le programme. Ses additions et multiplications sont calculées modulo* 2^{64}.

La composition permet de construire les programmes. Les équations $A_0 = \mathrm{id}_{\mathscr{U}}$ et $A_b \circ A_a = A_{a+b}$ permettent les premières optimisations : supprimer les additions de zéro et fusionner les additions adjacentes. Ce sont des propriétés de nos générateurs arithmétiques.

Côté optimiseur, on sépare la syntaxe du programme de sa sémantique. Dans cette première version, la syntaxe est une liste finie de symboles représentant les générateurs A_c et S, dans l'ordre d'exécution. Sa sémantique est le morphisme de $\mathscr{C}$ obtenu en composant ces fonctions. La liste vide représente l'identité. Par exemple, $[A_1, A_2]$ et $[A_3]$ sont deux descriptions différentes qui représentent la même fonction $x \mapsto x + 3$.

Pour un programme p, on note $[\![p]\!]$ son morphisme associé. Sa sémantique est donc une fonction, tandis que son évaluation sur une entrée $x \in \mathscr{U}$ est une valeur :

$$[\![p]\!] : \mathscr{U} \to \mathscr{U}, \qquad \mathrm{evaluate}(p, x) = [\![p]\!](x).$$

Par exemple, pour $p = [A_1, S]$, on a $[\![p]\!](x) = (x + 1)^2$ et $\mathrm{evaluate}(p, 3) = 16$. Une simplification correcte préserve la fonction :

$$[\![\mathrm{simplify}(p)]\!] = [\![p]\!],$$

c'est-à-dire le résultat pour toute entrée $x \in \mathscr{U}$.

On souhaite ensuite appliquer nos programmes à des listes de scalaires. Pour cela, on définit une catégorie $\mathscr{D}$ dont l'unique objet est $\mathrm{List}(\mathscr{U})$, l'ensemble des listes finies d'éléments de $\mathscr{U}$. Ses morphismes sont les fonctions obtenues en appliquant un morphisme de $\mathscr{C}$ à chaque élément d'une liste. On définit ainsi le foncteur $F : \mathscr{C} \to \mathscr{D}$

par $F(f)([x_1, \ldots, x_n]) = [f(x_1), \ldots, f(x_n)]$. La composition dans $\mathscr{D}$ est la composition usuelle des fonctions. Son identité est $F(\mathrm{id}_{\mathscr{U}}) = \mathrm{id}_{\mathrm{List}(\mathscr{U})}$, qui laisse chaque liste inchangée.

La loi de fonctorialité $F(g \circ f) = F(g) \circ F(f)$ permet de remplacer deux parcours successifs par un seul : chaque élément subit alors f, puis g, pendant le même parcours.

Exemple récapitulatif

Considérons les programmes $p = [A_0, A_1, A_2]$ et $q = [S]$. Composer p, puis q, construit le programme $r = [A_0, A_1, A_2, S]$, dont la sémantique est $[\![r]\!] = S \circ A_2 \circ A_1 \circ A_0$.

Les lois $A_0 = \mathrm{id}_{\mathscr{U}}$ et $A_2 \circ A_1 = A_3$ permettent de simplifier r en $r' = [A_3, S]$. Les programmes sont syntaxiquement différents, mais représentent la même fonction :

$$[\![r]\!](x) = [\![r']\!](x) = (x+3)^2 \pmod{2^{64}}.$$

Par exemple, leur évaluation sur 3 donne 36.

Reprenons ces mêmes programmes pour construire un programme de listes $P = [\mathrm{Map}(p), \mathrm{Map}(q)]$, où $\mathrm{Map}(p)$ signifie appliquer p à chaque élément. Sa sémantique est

$$[\![P]\!] = F(S) \circ F(A_2 \circ A_1 \circ A_0).$$

La fonctorialité permet de fusionner les deux étapes en $P_1 = [\mathrm{Map}(r)]$. Puis la simplification de r donne $P_2 = [\mathrm{Map}(r')]$. Concrètement, les programmes successifs sont :

$$\begin{aligned} P &= [\mathrm{Map}([A_0, A_1, A_2]), \mathrm{Map}([S])], \\ P_1 &= [\mathrm{Map}([A_0, A_1, A_2, S])], \\ P_2 &= [\mathrm{Map}([A_3, S])]. \end{aligned}$$

Comme dans l'exemple scalaire, la syntaxe change, mais la sémantique est conservée :

$$[\![P]\!] = [\![P_1]\!] = [\![P_2]\!] = F(S \circ A_3).$$

Sur l'entrée $[0, 1, 2]$, P effectue deux parcours :

$$[0,1,2] \xrightarrow{\mathrm{Map}(p)} [3,4,5] \xrightarrow{\mathrm{Map}(q)} [9,16,25].$$

Avec P_2, un seul parcours suffit : chaque élément x est directement transformé en $(x+3)^2$. On obtient encore $[9, 16, 25]$, sans construire la liste intermédiaire $[3, 4, 5]$ et avec une seule addition par élément.